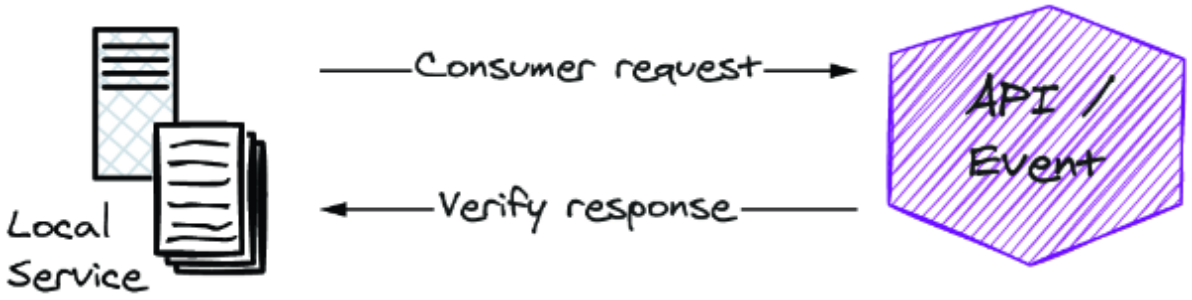


Figure 3.2 A provider replays the contract request and verifies the expected response against the local running service.

In a CDCT approach, the consumer will mock out the provider, and the provider mocks out their downstream dependencies. The provider must add a test pointing to the consumer within the contract testing process and will automatically get any updates if they point to the latest version of the consumer contract. Teams will discuss test failures; either the consumer changes their interaction, or the provider changes their implementation. In summary, a provider makes a service available to someone who needs it, replays requests for their service, and becomes an ambassador for contract testing across the organization.

3.3 What is a contract?

Now, we've spoken a lot about the contract that sits between the consumer and the provider. However, we haven't described precisely what the contract is yet. A *contract* is "a legal document that states and explains a formal agreement between two different people or groups" (https://mng.bz/lrBy). The same applies within software development, apart from being a legal document. However, some engineers may hold you to the same level of demands. In addition, there is another layer in the software—the application and service involved—but people or groups still author these, so it's a pretty good definition in this context.

What's important to differentiate is the difference between a contract and a schema. A *schema* is as "an outline of a plan or theory" (https://mng.bz/Dp5E). As the definition suggests, a schema provides the outline, which will be defined in the planning phase before development starts so that both parties work toward the same schema definition using tools such as JSON Schema (https://json-schema.org/). A schema tool